

Section-Based – Deployment (CDA)

1. Question

Category: CDA – Deployment

A serverless application, which is composed of multiple Lambda functions, has been deployed using AWS SAM. A developer was instructed to easily manage the deployments of the functions using CodeDeploy. When there is a new deployment, 10 percent of the incoming traffic should be shifted to the new version every 10 minutes until all traffic is shifted from the old version.

What should the developer do to properly deploy the functions that satisfies this requirement?

- Deploy the functions using a Canary deployment configuration.
- Deploy the functions using a Linear deployment configuration.
- Deploy the functions using an All-at-once deployment configuration.
- Deploy the functions using an Immutable deployment configuration.

Correct

CodeDeploy is a deployment service that automates application deployments to Amazon EC2 instances, on-premises instances, serverless Lambda functions, or Amazon ECS services. CodeDeploy can deploy application content that runs on a server and is stored in Amazon S3 buckets, GitHub repositories, or Bitbucket repositories. CodeDeploy can also deploy a serverless Lambda function. You do not need to make changes to your existing code before you can use CodeDeploy.

CodeDeploy supports the following deployment configurations:

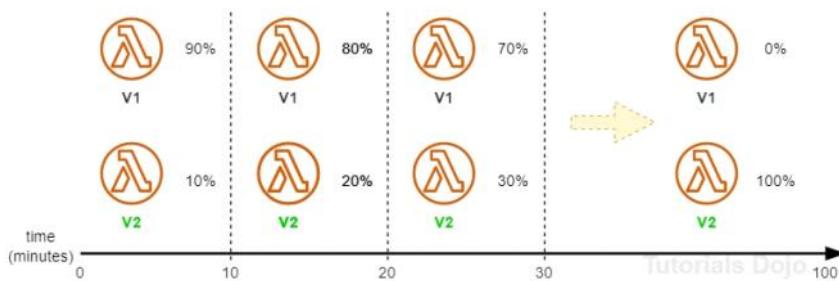
-**In-place (for EC2/On-premises)** – the application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated.

-**Canary (for Lambda/ECS)** – traffic is shifted in two increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Lambda function or ECS task set in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment.

-**Linear (for Lambda/ECS)** – traffic is shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.

-**All-at-once (for Lambda/ECS)** – all traffic is shifted from the original Lambda function or ECS task set to the updated function or task set all at once.

CodeDeployDefault.LambdaLinear10PercentEvery10Minutes



In a Linear deployment configuration, the traffic will be shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.

Hence, the correct answer is: **Deploy the functions using a Linear deployment configuration.**

The option that says: **Deploy the functions using a Canary deployment configuration** is incorrect because this will cause the traffic to be shifted in **two** increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Lambda function version in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment.

The option that says: **Deploy the functions using an All-at-once deployment configuration** is incorrect because, with this deployment configuration, the traffic is shifted from the original Lambda function to the updated Lambda function version all at once.

The option that says: **Deploy the functions using an Immutable deployment configuration** is incorrect because this is only applicable in Elastic Beanstalk and not to Lambda.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-configurations.html>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-codedeploy/>

2. Question

Category: CDA – Deployment

A developer plans to use AWS Elastic Beanstalk to deploy a microservice application. The application will be implemented in a multi-container Docker environment.

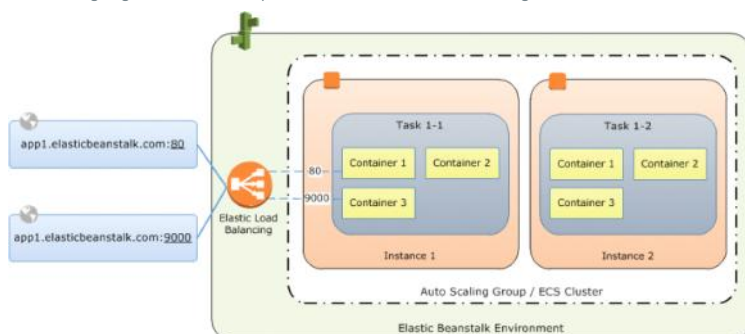
How should the developer configure the container definitions in the environment?

- Configure the container definitions in the Amazon ECS Console when building the Docker environment.
- Configure the container definitions in the `Dockerrun.aws.json` file.
- Use the `eb config` command to configure the container definitions.
- Configure the container definitions in the `Dockerrun.aws.json.config` and put it inside the `.ebextensions` folder.

Incorrect

Standard generic and preconfigured Docker platforms on Elastic Beanstalk support only a single Docker container per Elastic Beanstalk environment. In order to get the most out of Docker, Elastic Beanstalk lets you create an environment where your Amazon EC2 instances run multiple Docker containers side by side.

The following diagram shows an example Elastic Beanstalk environment configured with three Docker containers running on each Amazon EC2 instance in an Auto Scaling group:



Container instances—Amazon EC2 instances running Multicontainer Docker in an Elastic Beanstalk environment—require a configuration file named `Dockerrun.aws.json`. This file is specific to Elastic Beanstalk and can be used alone or combined with source code and content in a source bundle to create an environment on a Docker platform.

Hence, the correct answer is: **Configure the container definitions in the `Dockerrun.aws.json` file.**

The option that says: **Configure the container definitions in the Amazon ECS Console when building the Docker environment** is incorrect because the application must be deployed using Elastic Beanstalk. Therefore, you must configure the container definitions in the `Dockerrun.aws.json` file.

The option that says: **Configure the container definitions in the `Dockerrun.aws.json.config` and put it inside the `.ebextensions` folder** is incorrect because the `Dockerrun.aws.json` file should be placed on the same level where your application file resides.

The option that says: **Use the `eb config` command to configure the container definitions** is incorrect. This is just a command that you can use to change the configuration settings of your environment.

References:

https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/create_deploy_docker_ecstutorial.html
https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/create_deploy_docker_ecs.html
<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/single-container-docker-configuration.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdojo.com/aws-elastic-beanstalk/>

3. Question

Category: CDA – Deployment

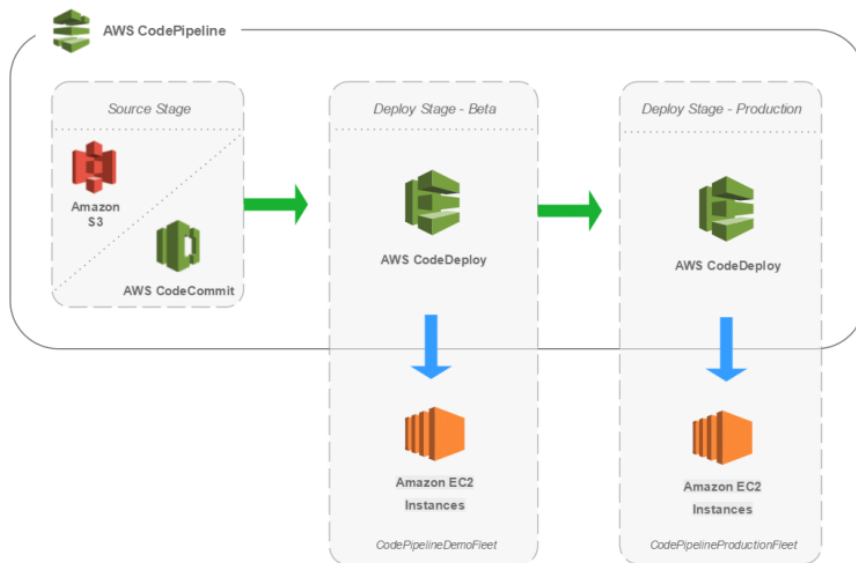
A company is heavily using a range of AWS services to host their enterprise applications. Currently, their deployment process still has a lot of manual steps which is why they plan to automate their software delivery process using continuous integration and delivery (CI/CD) pipelines in AWS. They will use CodePipeline to orchestrate each step of their release process and CodeDeploy for deploying applications to various compute platforms in AWS. In this architecture, which of the following are valid considerations when using CodeDeploy? (Select TWO.)

- You have to install and use the CodeDeploy agent installed on your EC2 instances and ECS cluster.
- The CodeDeploy agent communicates using HTTP over port 80.
- AWS Lambda compute platform deployments cannot use an in-place deployment type.
- CodeDeploy can deploy applications to EC2, AWS Lambda, and Amazon ECS only.
- CodeDeploy can deploy applications to both your EC2 instances as well as your on-premises servers.

Incorrect

CodeDeploy is a deployment service that automates application deployments to Amazon EC2 instances, on-premises instances, serverless Lambda functions, or Amazon ECS services.

CodeDeploy can deploy application content that runs on a server and is stored in Amazon S3 buckets, GitHub repositories, or Bitbucket repositories. CodeDeploy can also deploy a serverless Lambda function. You do not need to make changes to your existing code before you can use CodeDeploy.



CodeDeploy provides two deployment type options:

In-place deployment: The application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated. You can use a load balancer so that each instance is deregistered during its deployment and then restored to service after the deployment is complete. Only deployments that use the EC2/On-Premises compute platform can use in-place deployments. AWS Lambda compute platform deployments cannot use an in-place deployment type.

Blue/green deployment: The behavior of your deployment depends on which compute platform you use:

– **Blue/green on an EC2/On-Premises compute platform:** The instances in a deployment group (the original environment) are replaced by a different set of instances (the replacement environment). If you use an EC2/On-Premises compute platform, be aware that blue/green deployments work with Amazon EC2 instances only.

– **Blue/green on an AWS Lambda compute platform:** Traffic is shifted from your current serverless environment to one with your updated Lambda function versions. You can specify Lambda functions that perform validation tests and choose the way in which the traffic shift occurs. All AWS Lambda compute platform deployments are blue/green deployments. For this reason, you do not need to specify a deployment type.

– **Blue/green on an Amazon ECS compute platform:** Traffic is shifted from the task set with the original version of a containerized application in an Amazon ECS service to a replacement task set in the same service. The protocol and port of a specified load balancer listener are used to reroute production traffic. During deployment, a test listener can be used to serve traffic to the replacement task set while validation tests are run.

The CodeDeploy agent is a software package that, when installed and configured on an instance, makes it possible for that instance to be used in CodeDeploy deployments. The CodeDeploy agent communicates outbound using HTTPS over port 443.

It is also important to note that the CodeDeploy agent is required only if you deploy to an EC2/On-Premises compute platform. The agent is not required for deployments that use the Amazon ECS or AWS Lambda compute platform.

Therefore, the valid considerations in CodeDeploy in this scenario are:

– **AWS Lambda compute platform deployments cannot use an in-place deployment type.**

– **CodeDeploy can deploy applications to both your EC2 instances as well as your on-premises servers.**

The option that says: **CodeDeploy can deploy applications to EC2, AWS Lambda, and Amazon ECS only** is incorrect because it can also deploy to your on-premises servers.

The option that says: **The CodeDeploy agent communicates using HTTP over port 80** is incorrect because it is actually using HTTPS over port 443 and not HTTP.

The option that says: **You have to install and use the CodeDeploy agent installed on your EC2 instances and ECS cluster** is incorrect. Although this statement is true for EC2 instances, it is wrong for the latter as the CodeDeploy agent is not required for deployments that use the Amazon ECS or AWS Lambda compute platform.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html>
<https://docs.aws.amazon.com/codedeploy/latest/userguide/codedeploy-agent.html>
<https://aws.amazon.com/getting-started/projects/set-up-ci-cd-pipeline/>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-codedeploy/>

4. Question

Category: CDA – Deployment

An Elastic Beanstalk application becomes inaccessible for several minutes whenever a failed deployment is rolled back. A developer should recommend a strategy that will have the least impact on the application's availability if the deployment fails. Teams must be able to revert changes quickly as well.

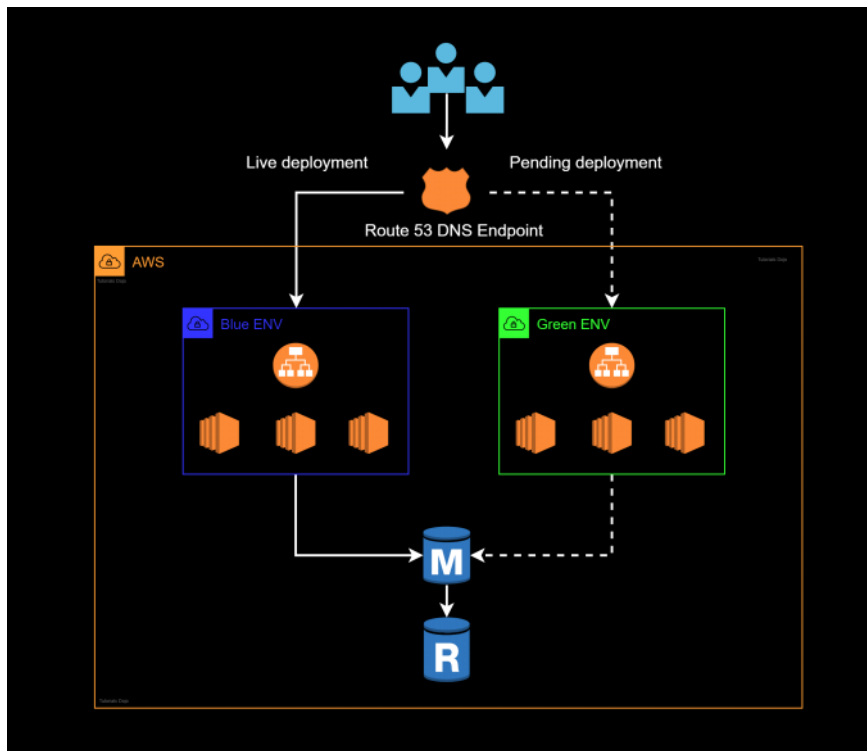
Which deployment method should the developer suggest?

- Blue/Green
- Rolling with Additional Batches
- Rolling
- All at Once

Correct

Because AWS Elastic Beanstalk performs an in-place update when you update your application versions, your application can become unavailable to users for a short period of time. You can avoid this downtime by performing a blue/green deployment, where you deploy the new version to a separate environment and then swap CNAMEs of the two environments to redirect traffic to the new version instantly.

Blue/green deployments require that your environment runs independently of your production database if your application uses one. If your environment has an Amazon RDS DB instance attached to it, the data will not transfer over to your second environment and will be lost if you terminate the original environment.



In Elastic Beanstalk, you can choose from a variety of deployment methods:

All at once – Deploy the new version to all instances simultaneously. All instances in your environment are out of service for a short time while the deployment occurs.

Rolling – Deploy the new version in batches. Each batch is taken out of service during the deployment phase, reducing your environment's capacity by the number of instances in a batch.

Rolling with additional batch – Deploy the new version in batches, but first launch a new batch of instances to ensure full capacity during the deployment process.

Immutable – Deploy the new version to a fresh group of instances by performing an immutable update.

Traffic splitting – Percentage of client traffic routed to new version temporarily impacted

Blue/Green – Deploy the new version to a separate environment, and then swap CNAMEs of the two environments to redirect traffic to the new version instantly.

The scenario is asking for the least impact on the application's availability if a deployment fails. Naturally, we'd want to roll back to the last working version if a deployment does not succeed. The rollback process for a Blue/green deployment is the fastest since all you have to do is switch back to the working environment's URL.

Hence, the correct answer is **Blue/Green**.

All at once is incorrect because this method deploys the new version to all instances simultaneously, which causes your instances to be out of service for a short time while the deployment occurs. This is also the case when you revert a failed deployment. In short, All at once has the MOST impact on your application's availability in case the deployment fails.

Rolling with additional batch is incorrect. Although this method ensures full capacity during deployment, its rollback process is quite slow because the deployment is done on fresh instances alongside the existing ones.

Rolling is incorrect. With Rolling, your environment's capacity to serve traffic is reduced by the number of instances the new version is being rolled out to, which may impact the availability of your application.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environmentmgmt-updates-immutable.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdojo.com/aws-elastic-beanstalk/>

5. Question

Category: CDA – Deployment

A Lambda function is over 80 MB in size, which exceeds the deployment package size limit for direct uploads. You want to refactor the function to pull in additional code and other dependencies from another source, which will reduce the size of the deployment.

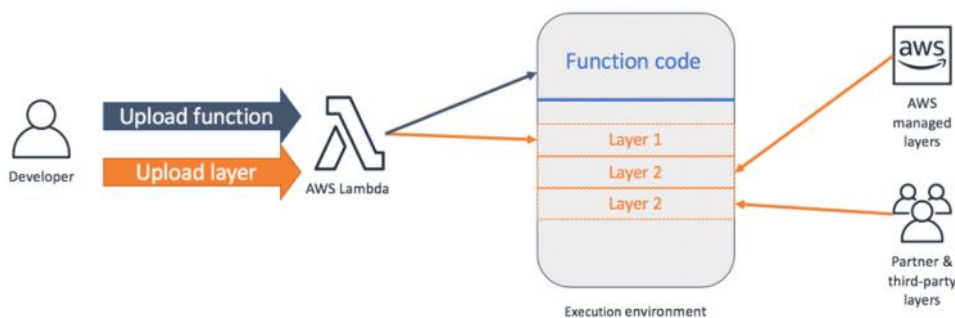
Which feature of Lambda should you use in order to implement the above task?

- Layers
- Environment Variable
- Execution Context
- Alias

Correct

You can configure your Lambda function to pull in additional code and content in the form of layers. A layer is a ZIP archive that contains libraries, a custom runtime, or other dependencies. With layers, you can use libraries in your function without needing to include them in your deployment package.

Layers let you keep your deployment package small, which makes development easier. You can avoid errors that can occur when you install and package dependencies with your function code. For Node.js, Python, and Ruby functions, you can develop your function code in the Lambda console as long as you keep your deployment package under 3 MB.



A function can use up to 5 layers at a time. The total unzipped size of the function and all layers can't exceed the unzipped deployment package size limit of 250 MB.

You can create layers, or use layers published by AWS and other AWS customers. Layers support resource-based policies for granting layer usage permissions to specific AWS accounts, AWS Organizations, or all accounts. Layers are extracted to the /opt directory in the function execution environment. Each runtime looks for libraries in a different location under /opt, depending on the language. Structure your layer so that function code can access libraries without additional configuration.

Hence, the correct answer is to use Lambda **Layers**.

Alias is incorrect because this is just like a pointer to a specific Lambda function version. This will not enable your function to pull in additional code and other dependencies from another source.

Execution Context is incorrect because this is a temporary runtime environment that initializes any external dependencies of your Lambda function code, such as database connections or HTTP endpoints.

Environment Variable is incorrect because this just enables you to dynamically pass settings to your function code and libraries, without making changes to your code.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/configuration-layers.html>
<https://docs.aws.amazon.com/lambda/latest/dg/limits.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

6. Question

Category: CDA – Deployment

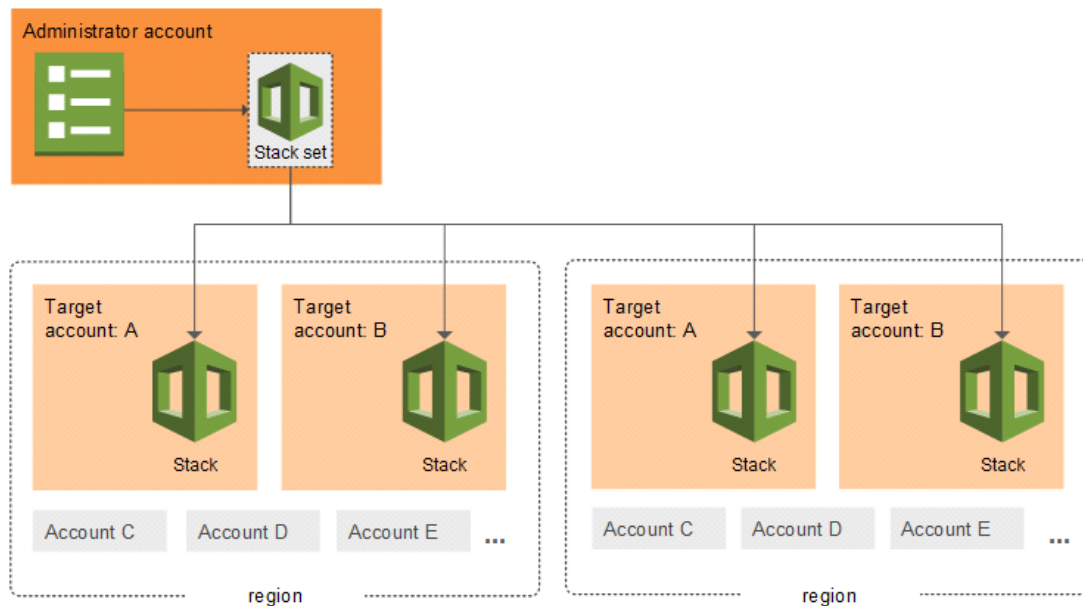
An application architect manages several AWS accounts for staging, testing, and production environments, which are used by several development teams. For application deployments, the developers use the similar base CloudFormation template for their applications.

Which of the following can allow the developer to effectively manage the updates on this template across all AWS accounts with minimal effort?

- Define and manage stack instances on multiple AWS Accounts using CloudFormation Stack Instances.
- Update the stacks on multiple AWS accounts using CloudFormation StackSets.
- Create and manage stacks on multiple AWS accounts using CloudFormation Change Sets.
- Use AWS CodePipeline to automate the deployment of CloudFormation templates across multiple accounts.

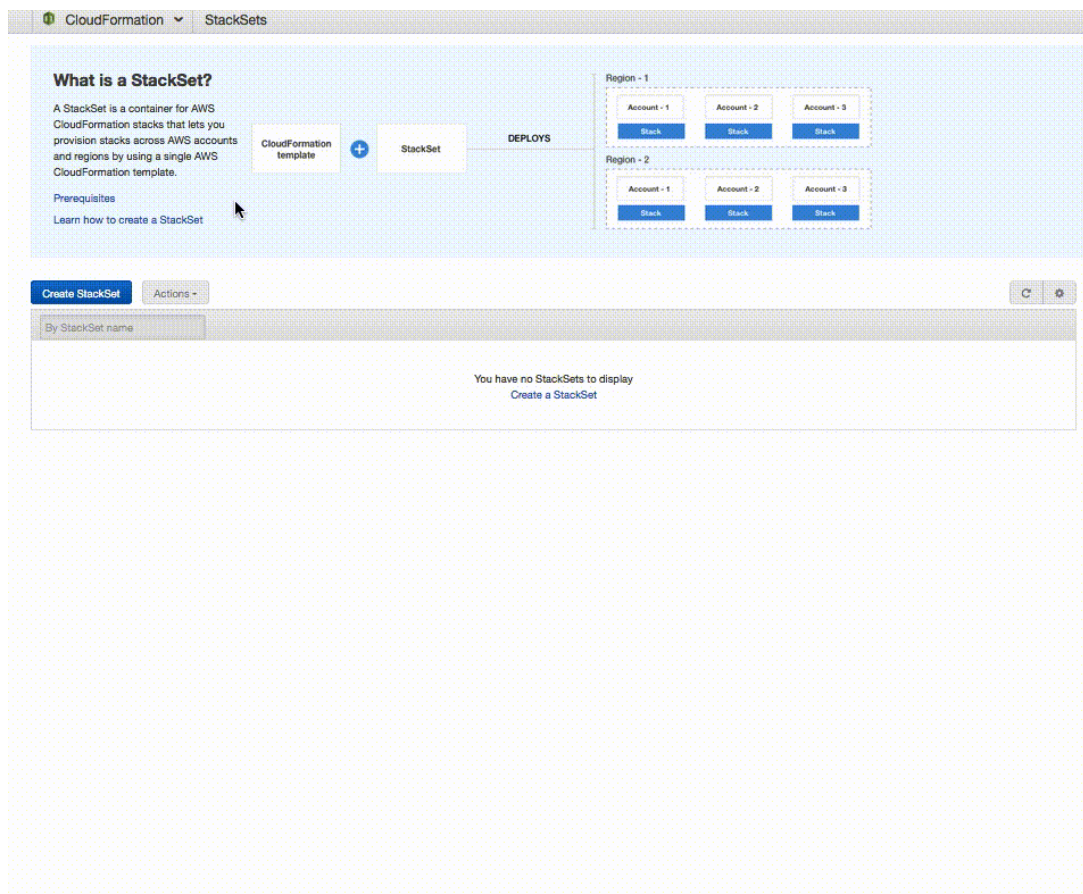
Correct

AWS CloudFormation StackSets extends the functionality of stacks by enabling you to create, update, or delete stacks across multiple accounts and regions with a single operation. Using an administrator account, you define and manage an AWS CloudFormation template, and use the template as the basis for provisioning stacks into selected target accounts across specified regions.



A **stack set** lets you create stacks in AWS accounts across regions by using a single AWS CloudFormation template. All the resources included in each stack are defined by the stack set's AWS CloudFormation template. As you create the stack set, you specify the template to use, as well as any parameters and capabilities that the template requires.

Hence, the correct solution in this scenario is to **update the stacks on multiple AWS accounts using CloudFormation StackSets**.



After you've defined a stack set, you can create, update, or delete stacks in the target accounts and regions you specify. When you create, update, or delete stacks, you can also specify operational preferences, such as the

order of regions in which you want the operation to be performed, the failure tolerance beyond which stack operations stop, and the number of accounts in which operations are performed on stacks concurrently. Remember that a stack set is a regional resource so if you create a stack set in one region, you cannot see it or change it in other regions.

The option that says: **Creating and managing stacks on multiple AWS accounts using CloudFormation Change Sets** is incorrect because Change Sets only allow you to preview how proposed changes to a stack might impact your running resources. In this scenario, the most suitable way to meet the requirement is to use StackSets.

The option that says: **Defining and managing stack instances on multiple AWS Accounts using CloudFormation Stack Instances** is incorrect because a stack instance is simply a reference to a stack in a target account within a region. Remember that a stack instance is associated with one stack set which is why this is just one of the components of CloudFormation StackSets.

The option that says: **Use AWS CodePipeline to automate the deployment of CloudFormation templates across multiple accounts** is incorrect. AWS CodePipeline can automate the deployment process, but it is primarily a CI/CD tool. While it can be configured to deploy CloudFormation templates, it does not inherently provide the same level of centralized management for multiple accounts and regions as StackSets does.

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/what-is-cfnstacksets.html>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacksets-getting-started.html>

Check out this AWS CloudFormation Cheat Sheet:
<https://tutorialsdojo.com/aws-cloudformation/>

7. Question

Category: CDA – Deployment

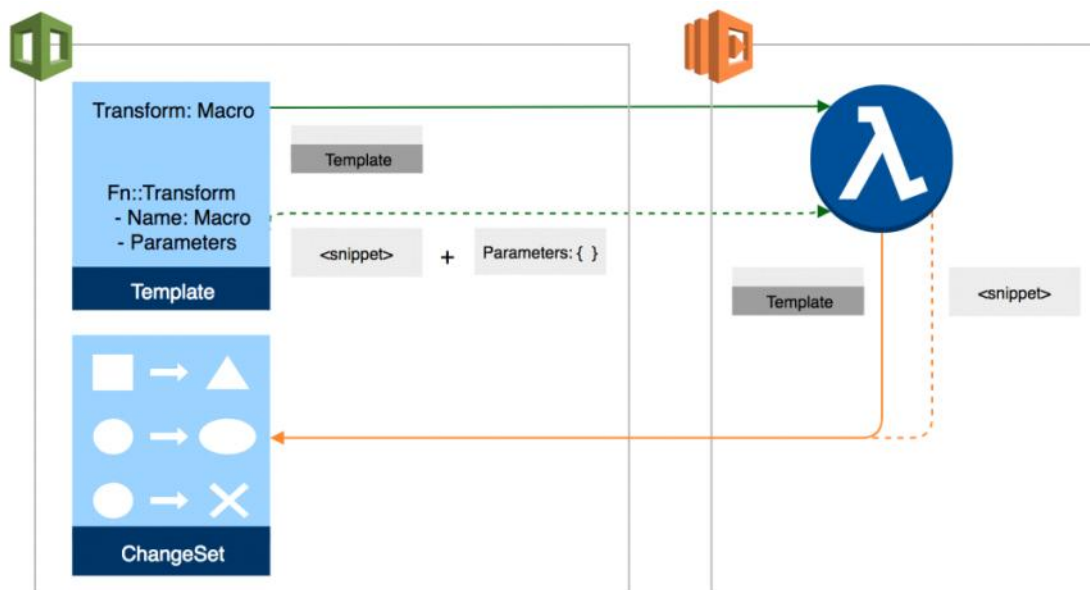
You are deploying a serverless application composed of Lambda, API Gateway, CloudFront, and DynamoDB using CloudFormation. The AWS SAM syntax should be used to declare resources in your template which requires you to specify the version of the AWS Serverless Application Model (AWS SAM).

Which of the following sections is required, aside from the `Resources` section, that should be in your CloudFormation template?

- Mappings
- Parameters
- Format Version
- Transform

Incorrect

For serverless applications (also referred to as Lambda-based applications), the optional **Transform** section specifies the version of the AWS Serverless Application Model (AWS SAM) to use. When you specify a transform, you can use AWS SAM syntax to declare resources in your template. The model defines the syntax that you can use and how it is processed.



This section specifies one or more macros that AWS CloudFormation uses to process your template. The Transform section builds on the simple, declarative language of AWS CloudFormation with a powerful macro system. You can declare one or more macros within a template. AWS CloudFormation executes macros in the order that they are specified. When you create a change set, AWS CloudFormation generates a change set that includes the processed template content. You can then review the changes and execute the change set.

AWS CloudFormation also supports the `AWS::Serverless` and `AWS::Include` transforms, which are macros hosted by AWS CloudFormation. AWS CloudFormation treats these transforms the same as any macros you create in terms of execution order and scope.

Therefore, the **Transform** section should be the correct one to be added to your template.

Mappings section is incorrect because this is just a literal mapping of keys and associated values that you can use to specify conditional parameter values, similar to a lookup table.

Parameters section is incorrect because this only contains the values that will be passed to your template at runtime (when you create or update a stack). You can refer to parameters from the `Resources` and `Outputs` sections of the template, but this is not used to specify the AWS SAM version.

Format Version section is incorrect because this just refers to the AWS CloudFormation template version that the template conforms to, and not the version of the AWS Serverless Application Model (AWS SAM)

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-anatomy.html>
<https://aws.amazon.com/blogs/aws/cloudformation-macros/>

Check out this AWS CloudFormation Cheat Sheet:
<https://tutorialsdojo.com/aws-cloudformation/>

Check out this AWS Serverless Application Model Cheat Sheet:
<https://tutorialsdojo.com/aws-serverless-application-model-sam/>

8. Question

Category: CDA – Deployment

Due to the popularity of serverless computing, your manager instructed you to share your technical expertise to the whole software development department of your company. You are planning to deploy a simple Node.js 'Hello World' Lambda function to AWS using CloudFormation.

Which of the following is the EASIEST way of deploying the function to AWS?

- Include your function source inline in the `Code` parameter of the `AWS::Lambda::Function` resource in the CloudFormation template.
- Upload the code in S3 then specify the `S3Key` and `S3Bucket` parameters under the `AWS::Lambda::Function` resource in the CloudFormation template.
- Upload the code in S3 as a ZIP file then specify the S3 path in the `ZipFile` parameter of the `AWS::Lambda::Function` resource in the CloudFormation template.
- Include your function source inline in the `ZipFile` parameter of the `AWS::Lambda::Function` resource in the CloudFormation template.

Incorrect

To create a Lambda function, you first create a Lambda function deployment package, a .zip or .jar file consisting of your code and any dependencies. When creating the zip, include only the code and its dependencies, not the containing folder. You will then need to set the appropriate security permissions for the zip package.

If you are using a CloudFormation template, you can configure the `AWS::Lambda::Function` resource which creates a Lambda function. To create a function, you need a deployment package and an execution role. The deployment package contains your function code. The execution role grants the function permission to use AWS services, such as Amazon CloudWatch Logs for log streaming and AWS X-Ray for request tracing.


```
ManagedPolicyArns:
  - arn:aws:iam::aws:policy/AWSLambdaExecute
  - arn:aws:iam::aws:policy/AmazonS3FullAccess
  - arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess
  - arn:aws:iam::aws:policy/AmazonKinesisFullAccess
Path: /
```

```
LambdaFunction:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName:
      Fn::Sub: lambda-function-${EnvName}
    Description: LambdaFunctioni of node 8
    Runtime: nodejs8.10
    Code:
      ZipFile:
        "exports.handler = function(event, context){\n
          console.log('Hello World By Tutorials Dojo');
        }"
    Handler: ${LambdaHandlerPath}
    MemorySize: 128
    Timeout: 10
    Role:
      Fn::GetAtt:
        - LambdaRole
        - Arn
    Environment:
      Variables:
        ENV:
          Fn::Sub: ${EnvName}
        TZ: UTC
```

Under the `AWS::Lambda::Function` resource, you can use the `Code` property which contains the deployment package for a Lambda function. For all runtimes, you can specify the location of an object in Amazon S3. For Node.js and Python functions, you can specify the function code inline in the template. Changes to a deployment package in Amazon S3 are not detected automatically during stack updates. To update the function code, change the object key or version in the template.

Hence, **including your function source inline in the `ZipFile` parameter of the `AWS::Lambda::Function` resource in the CloudFormation template** is the easiest way to deploy the Lambda function to AWS.

Uploading the code in S3 then specifying the `S3Key` and `S3Bucket` parameters under the `AWS::Lambda::Function` resource in the CloudFormation template is incorrect. Although this is a valid deployment step, you still have to upload the code in S3 instead of just including the function source inline in the `ZipFile` parameter. Take note that the scenario explicitly mentions that you have to pick the easiest way.

Including your function source inline in the `Code` parameter of the `AWS::Lambda::Function` resource in the CloudFormation template is incorrect because you should use the `ZipFile` parameter instead. Take note that the `Code` property is the parent property of the `ZipFile` parameter.

Uploading the code in S3 as a ZIP file then specifying the S3 path in the `ZipFile` parameter of the `AWS::Lambda::Function` resource in the CloudFormation template is incorrect because contrary to its name, the `ZipFile` parameter directly accepts the source code of your Lambda function and not an actual zip file. If you include your function source inline with this parameter, AWS CloudFormation places it in a file named `index` and `zips` it to create a deployment package. This is the reason why it is called the "`ZipFile`" parameter.

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-properties-lambda-function-code.html>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-lambda-function.html>

Check out this AWS CloudFormation Cheat Sheet:

<https://tutorialsdojo.com/aws-cloudformation/>

9. Question

Category: CDA – Deployment

A developer is building an online game in AWS which will be using a NoSQL database with DynamoDB. Each player data has an average size of 3.5 KB and it is expected that the game will send 150 eventually consistent read requests per second.

How many Read Capacity Units (RCU) should the developer provision to the table?

- 300
- 75
- 150
- 600

Incorrect

One *read request unit* represents one strongly consistent read request, or two eventually consistent read requests, for an item up to 4 KB in size. Transactional read requests require 2 read request units to perform one read for items up to 4 KB.

If you need to read an item that is larger than 4 KB, DynamoDB needs additional read request units. The total number of read request units required depends on the item size, and whether you want an eventually consistent or strongly consistent read.

Capacity calculator

Avg. item size

4

KB

Item read/sec

150

Eventually consistent

Item write/sec

1

Standard

Read capacity

75

Write capacity

4

Estimated cost

\$9.19 / month

per table/index

Cancel

Update

For example, suppose that you create a table with 10 provisioned read capacity units. This allows you to perform 10 strongly consistent reads per second, or 20 eventually consistent reads per second, for items up to 4 KB. Reading an item larger than 4 KB consumes more read capacity units. For example, a strongly consistent read of an item that is 8 KB (4 KB × 2) consumes 2 read capacity units. An eventually consistent read on that same item consumes only 1 read capacity unit. Item sizes for reads are rounded up to the next 4 KB multiple. For example, reading a 3,500-byte item consumes the same throughput as reading a 4 KB item. To get the number of RCU required to handle 150 eventually consistent read requests with an average item size of 3.5 KB, you simply have to do the following steps:

Step #1 Get the Average Item Size by rounding up to 4 KB

= 3.5 KB = 4 KB (rounded up)

Step #2 Get the RCU per Item by dividing the Average Item Size by 8 KB

= 4 KB / 8 KB

= 0.5

Step #3 Multiply the RCU per item to the number of items to be written per second

= 150 × 0.5

= 75 eventually consistent read requests

Hence, the correct answer is **75**.

150 is incorrect because this is the value for strongly consistent read requests based on the given RCU. Take note that for Step #2, you have to divide the Average Item Size by 8 KB and not by 4 KB, if you are calculating for eventual consistency.

300 is incorrect because this is the value for transactional read requests based on the given RCU. Take note that for Step #2, you have to divide the Average Item Size by 8 KB and not by 2 KB, if you are calculating for eventual consistency.

600 is incorrect because this is the value for transactional **write** requests, which is irrelevant since only the RCU is provided in the scenario.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ProvisionedThroughput.html#ItemSizeCalculations.Reads>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

Calculating the Required Read and Write Capacity Unit for your DynamoDB Table:

<https://tutorialsdojo.com/calculating-the-required-read-and-write-capacity-unit-for-your-dynamodb-table/>

10. Question

Category: CDA – Deployment

An aerospace engineering company has recently migrated to AWS for their cloud architecture. They are using CloudFormation and AWS SAM as deployment services for both of their monolithic and serverless applications. There is a new requirement where you have to dynamically install packages, create files, and start services on your EC2 instances upon the deployment of the application stack using CloudFormation.

Which of the following helper scripts should you use in this scenario?

- cfn-hup
- cfn-init
- cfn-signal
- cfn-get-metadata

Correct

AWS CloudFormation provides the following Python helper scripts that you can use to install software and start services on an Amazon EC2 instance that you create as part of your stack:

[cfn-init](#): Use to retrieve and interpret resource metadata, install packages, create files, and start services.

[cfn-signal](#): Use to signal with a CreationPolicy or WaitCondition, so you can synchronize other resources in the stack when the prerequisite resource or application is ready.

[cfn-get-metadata](#): Use to retrieve metadata for a resource or path to a specific key.

[cfn-hup](#): Use to check for updates to metadata and execute custom hooks when changes are detected.

You call the scripts directly from your template. The scripts work in conjunction with resource metadata that's defined in the same template. The scripts run on the Amazon EC2 instance during the stack creation process.

The scripts are not executed by default. You must include calls in your template to execute specific helper scripts.

```

35     "Ec2Instance" : {
36       "Type" : "AWS::EC2::Instance",
37       "Metadata" : {
38         "AWS::CloudFormation::Init" : {
39           "configSets" : {
40             "config" : [
41               "rename",
42               "bootstrapDSC"
43             ]
44           },
45           "rename" : {
46             "a-rename-computer" : {
47               "command" : "powershell.exe -Command Rename-Computer -NewName Server1 -Restart",
48               "waitAfterCompletion" : "forever"
49             },
50             "bootstrapDSC" : {
51               "a-setpullmode" : {
52                 "command" : "powershell.exe -Command c:\\cfn\\scripts\\SetPullmode.ps1",
53                 "waitAfterCompletion" : "g"
54               }
55             }
56           }
57         },
58       },
59       "Properties" : { ...
74     },
75   },

```

Hence, **cfn-init** helper script is the correct answer since it interprets the metadata that contains the sources, packages, files, and services. You run the script on the EC2 instance when it is launched. The script is installed by default on Amazon Linux and Windows AMIs.

The **cfn-get-metadata** helper script is incorrect since it is only a wrapper script that retrieves either all metadata that is defined for a resource or path to a specific key or a subtree of the resource metadata, but does not interpret the resource metadata, install packages, create files, and start services.

The **cfn-signal** helper script is incorrect since it does not perform any retrieval and interpretation of resource metadata, installation of packages, creation of files, and starting of services. Instead, it is a wrapper that signals an AWS CloudFormation WaitCondition for synchronizing other resources in the stack when the application is ready.

The **cfn-hup** helper script is incorrect because this is just a daemon that checks for updates to metadata and executes custom hooks when changes are detected. It does not retrieve and interpret the resource metadata, install packages, create files, and start services unlike **cfn-init** helper script.

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/cfn-init.html>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/cfn-helper-scripts-reference.html>
<https://s3.amazonaws.com/cloudformation-examples/BootstrappingApplicationsWithAWSCloudFormation.pdf>

Check out this AWS CloudFormation Cheat Sheet:

<https://tutorialsdojo.com/aws-cloudformation/>

AWS CloudFormation – Templates, Stacks, Change Sets:

11. Question

Category: CDA – Deployment

A developer is preparing the application specification (AppSpec) file in CodeDeploy, which will be used to deploy her Lambda functions to AWS. In the deployment, she needs to configure CodeDeploy to run a task before the traffic is shifted to the deployed Lambda function version.

Which deployment lifecycle event should she configure in this scenario?

- BeforeAllowTraffic
- BeforeInstall
- Start
- Install

Correct

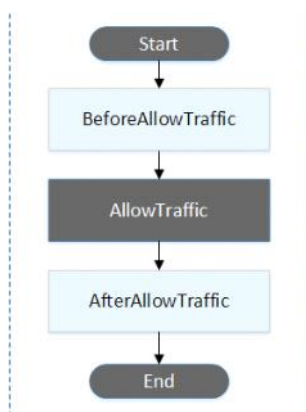
The content in the 'hooks' section of the AppSpec file varies, depending on the compute platform for your deployment. The 'hooks' section for an EC2/On-Premises deployment contains mappings that link deployment lifecycle event hooks to one or more scripts. The 'hooks' section for a Lambda or an Amazon ECS deployment specifies Lambda validation functions to run during a deployment lifecycle event. If an event hook is not present, no operation is executed for that event. This section is required only if you are running scripts or Lambda validation functions as part of the deployment.

An AWS Lambda hook is one Lambda function specified with a string on a new line after the name of the lifecycle event. Each hook is executed once per deployment. Following are descriptions of the hooks that are available for use in your AppSpec file.

BeforeAllowTraffic – Use to run tasks before traffic is shifted to the deployed Lambda function version.

AfterAllowTraffic – Use to run tasks after all traffic is shifted to the deployed Lambda function version.

In a serverless Lambda function version deployment, event hooks run in the following order:



Take note that **Start**, **AllowTraffic**, and **End** events in the deployment cannot be scripted, which is why they appear in gray in the above diagram.

Hence, the correct answer is **BeforeAllowTraffic**.

Start is incorrect because this deployment lifecycle event in Lambda cannot be scripted or used. In this scenario, the correct event that you should configure is the **BeforeAllowTraffic** event.

BeforeInstall is incorrect because this event is only applicable for ECS, EC2 or On-Premises compute platforms and not for Lambda deployments.

Install is incorrect because this uses the CodeDeploy agent to copy the revision files from the temporary location to the final destination folder of the EC2 or On-Premises server. This deployment lifecycle event is not available for Lambda as well as for ECS deployments.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/reference-appspec-file-structure-hooks.html#appspec-hooks-lambda>
<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-steps.html#deployment-steps-lambda>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-codedeploy/>

12. Question

Category: CDA – Deployment

The current application deployment process of a company is tedious and is prone to errors. They asked a developer to set up CodeDeploy as their deployment service, which can automate their application deployments on their

hybrid cloud architecture.

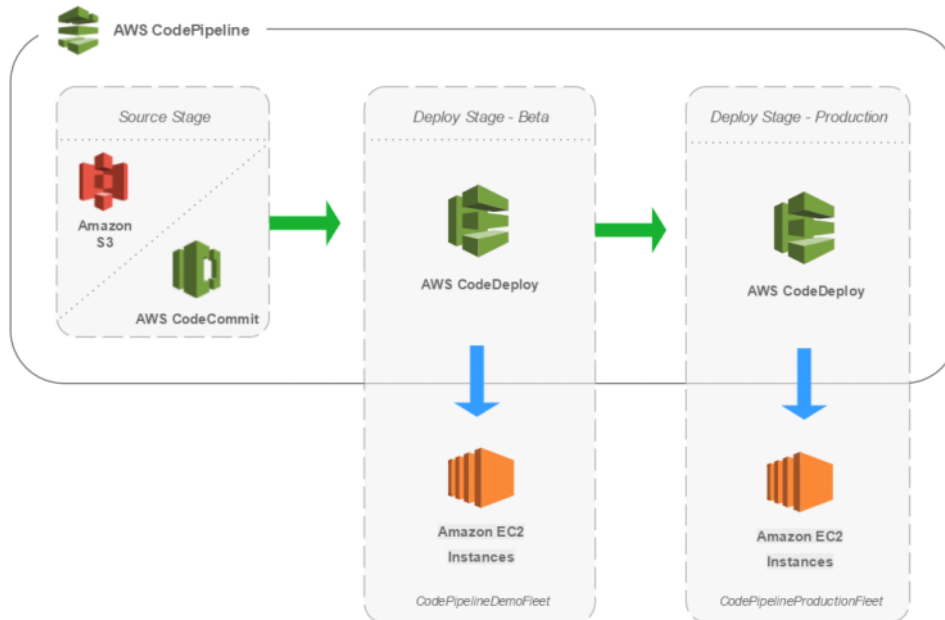
Which of the following deployment types does CodeDeploy support? (Select TWO.)

- In-place deployments to on-premises servers
- Blue/green deployments to ECS.
- In-place deployments to AWS Lambda.
- Blue/green deployments to on-premises servers.
- Rolling deployments to ECS.

Correct

CodeDeploy is a deployment service that automates application deployments to Amazon EC2 instances, on-premises instances, serverless Lambda functions, or Amazon ECS services.

CodeDeploy can deploy application content that runs on a server and is stored in Amazon S3 buckets, GitHub repositories, or Bitbucket repositories. CodeDeploy can also deploy a serverless Lambda function. You do not need to make changes to your existing code before you can use CodeDeploy.



CodeDeploy provides two deployment type options:

In-place deployment: The application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated. You can use a load balancer so that each instance is deregistered during its deployment and then restored to service after the deployment is complete. Only deployments that use the EC2/On-Premises compute platform can use in-place deployments. AWS Lambda compute platform deployments cannot use an in-place deployment type.

Blue/green deployment: The behavior of your deployment depends on which compute platform you use:

– **Blue/green on an EC2/On-Premises compute platform:** The instances in a deployment group (the original environment) are replaced by a different set of instances (the replacement environment). If you use an EC2/On-Premises compute platform, be aware that blue/green deployments work with Amazon EC2 instances only.

– **Blue/green on an AWS Lambda compute platform:** Traffic is shifted from your current serverless environment to one with your updated Lambda function versions. You can specify Lambda functions that perform validation tests and choose the way in which the traffic shift occurs. All AWS Lambda compute platform deployments are blue/green deployments. For this reason, you do not need to specify a deployment type.

– **Blue/green on an Amazon ECS compute platform:** Traffic is shifted from the task set with the original version of a containerized application in an Amazon ECS service to a replacement task set in the same service. The protocol and port of a specified load balancer listener are used to reroute production traffic. During deployment, a test listener can be used to serve traffic to the replacement task set while validation tests are run.

The CodeDeploy agent is a software package that, when installed and configured on an instance, makes it possible for that instance to be used in CodeDeploy deployments. The CodeDeploy agent communicates outbound using HTTPS over port 443.

It is also important to note that the CodeDeploy agent is required only if you deploy to an EC2/On-Premises compute platform. The agent is not required for deployments that use the Amazon ECS or AWS Lambda compute platform.

Therefore, the supported deployment types in CodeDeploy are:

– **Blue/green deployments to ECS**

– **In-place deployments to on-premises servers**

Rolling deployments to ECS is incorrect because only blue/green deployment is allowed if you used the AWS CodeDeploy service to deploy the new version of your application to ECS. Take note that in CodeDeploy, only the EC2/On-Premises compute platform can use both in-place deployments and blue/green deployment. For Lambda and ECS, you can only do a blue/green deployment in CodeDeploy. This type of deployment is actually done in Elastic Beanstalk for Multicontainer docker environment which implicitly uses ECS.

In-place deployments to AWS Lambda is incorrect because AWS Lambda compute platform deployments cannot use an in-place deployment type.

Blue/green deployments to on-premises servers is incorrect because, in CodeDeploy, blue/green deployments only work with Amazon EC2 instances only.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/codedeploy-agent.html>

<https://aws.amazon.com/getting-started/projects/set-up-ci-cd-pipeline/>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-codedeploy/>

Elastic Beanstalk vs CloudFormation vs OpsWorks vs CodeDeploy:

<https://tutorialsdojo.com/elastic-beanstalk-vs-cloudformation-vs-opsworks-vs-codedeploy/>

Comparison of AWS Services Cheat Sheets:

<https://tutorialsdojo.com/comparison-of-aws-services/>

13. Question

Category: CDA – Deployment

A company is deploying the package of its Lambda function, which is compressed as a ZIP file, to AWS. However, they are getting an error in the deployment process because the package is too large. The manager instructed the developer to keep the deployment package small to make the development process much easier and more modularized. This should also help prevent errors that may occur when dependencies are installed and packaged with the function code.

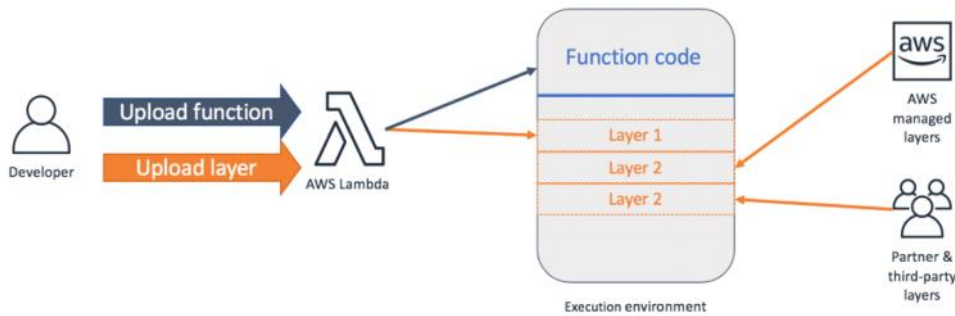
Which of the following options is the MOST suitable solution that the developer should implement?

- Upload the deployment package to S3.
- Upload the other dependencies of your function as a separate Lambda Layer instead.
- Zip the deployment package again to further compress the zip file.
- Compress the deployment package as TAR file instead.

Correct

You can configure your Lambda function to pull in additional code and content in the form of layers. A layer is a ZIP archive that contains libraries, a [custom runtime](#), or other dependencies. With layers, you can use libraries in your function without needing to include them in your deployment package.

Layers let you keep your deployment package small, which makes development easier. You can avoid errors that can occur when you install and package dependencies with your function code. For Node.js, Python, and Ruby functions, you can [develop your function code in the Lambda console](#) as long as you keep your deployment package under 3 MB.



A function can use up to 5 layers at a time. The total unzipped size of the function and all layers can't exceed the unzipped deployment package size limit of 250 MB.

You can create layers, or use layers published by AWS and other AWS customers. Layers support resource-based policies for granting layer usage permissions to specific AWS accounts, AWS Organizations, or all accounts. Layers are extracted to the `/opt` directory in the function execution environment. Each runtime looks for libraries in a different location under `/opt`, depending on the language. Structure your layer so that function code can access libraries without additional configuration.

Hence, the correct answer is to **upload the other dependencies of your function as a separate Lambda Layer instead.**

Uploading the deployment package to S3 is incorrect. Although you can upload large deployment packages of over 50 MB in size via S3, your function will still be in a single layer. This doesn't meet the requirement of making the deployment package small and modularized. You have to use Lambda Layers instead.

Zippping the deployment package again to further compress the zip file is incorrect because doing this will not significantly make the ZIP file smaller.

Compressing the deployment package as TAR file instead is incorrect. Although it may decrease the size of the deployment package, it is still not enough to totally solve the issue. A compressed TAR file is not significantly smaller as compared to a ZIP file.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/configuration-layers.html>

<https://docs.aws.amazon.com/lambda/latest/dg/limits.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

14. Question

Category: CDA – Deployment

A Lambda function has been integrated with DynamoDB Streams as its event source. There has been a new version of the function that needs to be deployed using CodeDeploy where the traffic must be shifted in two increments. It should shift 10 percent of the incoming traffic to the new version in the first increment and then the remaining 90 percent should be deployed five minutes later.

Which of the following deployment configurations is the MOST suitable to satisfy this requirement?

- Linear
- Rolling with additional batch
- All-at-once
- Canary

Correct
CodeDeploy is a deployment service that automates application deployments to Amazon EC2 instances, on-premises instances, serverless Lambda functions, or Amazon ECS services. CodeDeploy can deploy application content that runs on a server and is stored in Amazon S3 buckets, GitHub repositories, or Bitbucket repositories. CodeDeploy can also deploy a serverless Lambda function. You do not need to make changes to your existing code before you can use CodeDeploy.

CodeDeploy supports the following deployment configurations:

-In-place (for EC2/On-premises) – the application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated.

-Canary (for Lambda/ECS) – traffic is shifted in two increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Lambda function or ECS task set in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment.

-Linear (for Lambda/ECS) – traffic is shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.

-All-at-once (for Lambda/ECS) – all traffic is shifted from the original Lambda function or ECS task set to the updated function or task set all at once.

CodeDeployDefault.LambdaCanary10Percent5Minutes



In a **Canary** deployment configuration, the traffic is shifted in **two** increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Lambda function version in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment. Hence, this is the correct answer which will satisfy the requirement for the given scenario.

Linear is incorrect because this will cause the traffic to be shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.

All-at-once is incorrect because with this deployment configuration, the traffic is shifted from the original Lambda function to the updated Lambda function version all at once.

Rolling with additional batch is incorrect because this is only applicable in Elastic Beanstalk and not for Lambda.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-configurations.html>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-codedeploy/>

15. Question

Category: CDA – Deployment

A development team is working on an AWS Serverless Application Model (SAM) application with its source code hosted on GitHub. A newly recruited developer clones the repository and observes that the SAM template contains references to AWS Lambda functions with `CodeUri` pointing to local file paths. The developer has added a new Lambda function and must redeploy the updated version to Production.

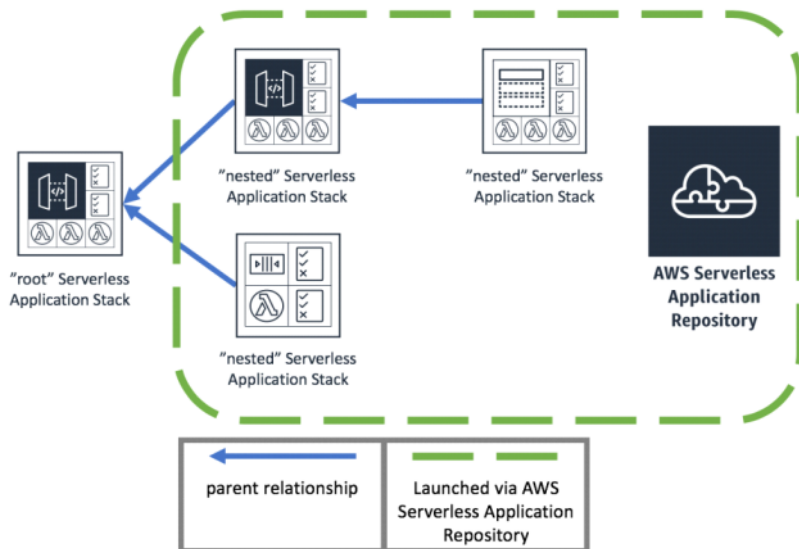
Which combination of steps must be taken to satisfy the requirement? (Select Two)

- Execute `sam build` to resolve dependencies and construct deployment artifacts for all functions and layers in the SAM template.

- Use the `sam sync` command to synchronize the local changes to the application in AWS.
- Run `sam init` to initialize a new SAM project.
- Execute `sam publish` to make the application available in the AWS Serverless Application Repository.
- Use the `sam deploy` command to deploy the application with a specified CloudFormation stack.

Incorrect

AWS SAM uses AWS CloudFormation as the underlying deployment mechanism. You can deploy your application by using AWS SAM command line interface (CLI) commands. You can also use other AWS services that integrate with AWS SAM to automate your deployments.



When deploying a SAM application, it's vital to ensure that all components of the application are properly packaged and that all resources are provisioned correctly in the AWS environment. The `sam build` command serves this purpose by resolving any dependencies the application might have and constructing deployment artifacts for all functions and layers specified in the SAM template. This is especially important when the SAM template references local file paths, such as `CodeUri` pointing to local Lambda function codes.

Once the application has been successfully built, the next step is to deploy it. The `sam deploy` command allows the application to be deployed using AWS CloudFormation, ensuring that all resources defined in the SAM template are provisioned and configured correctly in the target environment.

Hence, the correct answers are:

- Execute `sam build` to resolve dependencies and construct deployment artifacts for all functions and layers in the SAM template.
- Use the `sam deploy` command to deploy the application with a specified CloudFormation stack.

The option that says: **Run `sam init` to initialize a new SAM project** is incorrect because this command is simply used when creating a new SAM project, not to deploy an existing one.

The option that says: **Execute `sam publish` to make the application available in the AWS Serverless Application Repository** is incorrect. This SAM CLI is used to publish applications to the AWS Serverless Application Repository, which is not a mandatory step for deploying SAM applications.

The option that says: **Use the `sam sync` command to synchronize the local changes to the application in AWS** is incorrect. This command is typically used for quick syncing of local changes to AWS and is more suitable for rapid development testing. In a Production setting, a full deployment using `sam deploy` is more appropriate to ensure that all configurations and resources are correctly provisioned.

References:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-deploying.html>
<https://docs.aws.amazon.com/serverlessrepo/latest/devguide/what-is-serverlessrepo.html>

Check out this AWS SAM Cheat Sheet:

<https://tutorialsdojo.com/aws-serverless-application-model-sam/>

16. Question

Category: CDA – Deployment

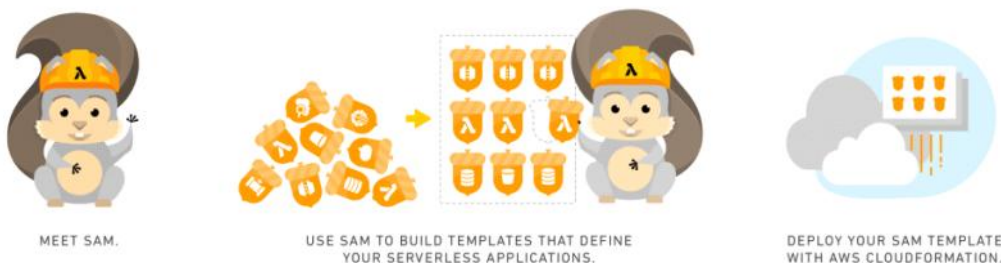
A developer is instructed to set up a new serverless architecture composed of AWS Lambda, API Gateway, and DynamoDB in a single stack. The new architecture should allow the developer to locally build, test, and debug serverless applications.

Which of the following should the developer use to satisfy the above requirement?

- AWS Systems Manager
- AWS Serverless Application Model (AWS SAM)
- AWS Elastic Beanstalk
- AWS CloudFormation

Correct

The AWS Serverless Application Model (AWS SAM) is an open-source framework that you can use to build serverless applications on AWS. It consists of the AWS SAM template specification that you use to define your serverless applications and the AWS SAM command line interface (AWS SAM CLI) that you use to build, test, and deploy your serverless applications.



Because AWS SAM is an extension of AWS CloudFormation, you get the reliable deployment capabilities of AWS CloudFormation. You can define resources by using AWS CloudFormation in your AWS SAM template. Also, you can use the full suite of resources, intrinsic functions, and other template features that are available in AWS CloudFormation.

You can use AWS SAM with a suite of AWS tools for building serverless applications. The AWS SAM CLI lets you locally build, test, and debug serverless applications that are defined by AWS SAM templates. The CLI provides a Lambda-like execution environment locally. It helps you catch issues upfront by providing parity with the actual Lambda execution environment. To step through and debug your code to understand what the code is doing, you can use AWS SAM with AWS toolkits like the AWS Toolkit for JetBrains, AWS Toolkit for PyCharm, AWS Toolkit for IntelliJ, and AWS Toolkit for Visual Studio Code. This tightens the feedback loop by making it possible for you to find and troubleshoot issues that you might run into in the cloud.

Therefore, the most suitable service to use in this scenario is **AWS Serverless Application Model (AWS SAM)**.

AWS CloudFormation is incorrect. Although this service can certainly be used to deploy Lambda, API Gateway, DynamoDB, and other AWS resources of your serverless application, it doesn't have the capability to locally build, test, and debug your application like what AWS SAM has. In addition, AWS SAM is a more suitable service to use if you want to deploy and manage your serverless applications in AWS just as mentioned above.

AWS Systems Manager is incorrect because this service is primarily used for managing resources in your AWS environment, not for building, testing, and debugging serverless applications.

AWS Elastic Beanstalk is incorrect because this service is not suitable for deploying serverless applications. In addition, it doesn't have the capability to locally build, test, and debug your serverless applications as effectively as what AWS SAM can do.

References:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html>
<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/automating-updates-to-serverless-apps.html>

Check out this AWS Serverless Application Model Cheat Sheet:
<https://tutorialsdojo.com/aws-serverless-application-model-sam/>

Comparison of AWS Services Cheat Sheets:
<https://tutorialsdojo.com/comparison-of-aws-services/>

17. Question

Category: CDA – Deployment

An application has recently been migrated from an on-premises data center to a development Elastic Beanstalk environment. A developer will do iterative tests and therefore needs to deploy code changes and view them as quickly as possible.

Which of the following options take the LEAST amount of time to complete the deployment?

- Rolling with additional batch
- All at once
- Rolling
- Immutable

Incorrect

In ElasticBeanstalk, you can choose from a variety of deployment methods:

-**All at once** – Deploy the new version to all instances simultaneously. All instances in your environment are out of service for a short time while the deployment occurs. This is the method that provides the least amount of time for deployment.

-**Rolling** – Deploy the new version in batches. Each batch is taken out of service during the deployment phase, reducing your environment’s capacity by the number of instances in a batch.

-**Rolling with additional batch** – Deploy the new version in batches, but first launch a new batch of instances to ensure full capacity during the deployment process.

-**Immutable** – Deploy the new version to a fresh group of instances by performing an [immutable update](#).

-**Blue/Green** – Deploy the new version to a separate environment, and then swap CNAMEs of the two environments to redirect traffic to the new version instantly.

Refer to the table below for the characteristics of each deployment method as well as the amount of time it takes to do the deployment, as seen in the **Deploy Time** column:

Method	Impact of Failed Deployment	Deploy Time	Zero Downtime	No DNS Change	Rollback Process	Code Deployed To
All at once	Downtime	☹	x	✓	Manual Redeploy	Existing instances
Rolling	Single batch out of service; any successful batches before failure running new application version	☹ ☹ †	✓	✓	Manual Redeploy	Existing instances
Rolling with additional batch	Minimal if first batch fails; otherwise, similar to Rolling	☹ ☹ ☹ †	✓	✓	Manual Redeploy	New and existing instances
Immutable	Minimal	☹ ☹ ☹ ☹	✓	✓	Terminate New Instances	New instances
Blue/green	Minimal	☹ ☹ ☹ ☹	✓	x	Swap URL	New instances

Hence, the correct answer in this scenario is **All at once**.

Rolling is incorrect because this will deploy the new version in batches only to existing instances, without provisioning new resources.

Immutable is incorrect because this will deploy the new version to a fresh group of instances by performing an immutable update. Considering the time to deploy additional instances and installing the new version, this option does not provide the least amount of deployment time.

Rolling with additional batch is incorrect because this just deploys the new version in batches. It first launches a new batch of instances to ensure full capacity during the deployment process then proceeds in deploying the new version.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>
<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.CNAMEswap.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:
<https://tutorialsdojo.com/aws-elastic-beanstalk/>

18. Question

Category: CDA – Deployment

An EBS-backed EC2 instance has been recently reported to contain a malware that could spread to your other instances. To fix this security vulnerability, you will need to attach its root EBS volume to a new EC2 instance which hosts a security program that can scan viruses, worms, Trojan horses, or spyware.

What steps would you take to detach the root volume from the compromised EC2 instance?

- Unmount the volume from the OS and then detach.
- Stop the instance then detach the volume.
- Unmount the volume, stop the instance, and then detach.
- Detach the volume from the AWS Console. AWS takes care of unmounting the volume for you.

Correct

You can detach an Amazon EBS volume from an instance explicitly or by terminating the instance. However, if the instance is running, you must first unmount the volume from the instance.

If an EBS volume is the root device of an instance, you must **stop the instance before you can detach the volume**.

The options that say **unmount the volume from the OS and then detach** and **unmount the volume, stop the instance, and then detach** are both incorrect because you can’t unmount the root volume on a running instance.

The option that says: **Detach the volume from the AWS Console. AWS takes care of unmounting the volume for you** is incorrect because unmounting the volume is not managed by AWS.

Reference:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-detaching-volume.html>

Check out this Amazon EC2 Cheat Sheet:
<https://tutorialsdojo.com/amazon-elastic-compute-cloud-amazon-ec2/>

19. Question

Category: CDA – Deployment

An application is hosted in Elastic Beanstalk, which is currently running in Java 7 runtime environment. A new version of the application is ready to be deployed, and the developer was tasked to upgrade the platform to Java 8 to accommodate the changes. All user traffic must be immediately directed to the new version. If problems arise, the developer should be able to quickly revert to the previous version.

Which of the following is the MOST appropriate action that the developer should do to upgrade the platform?

- Perform a Traffic splitting deployment.
- Perform a Blue/Green Deployment.
- Update the environment's platform version to Java 8.
- Manually upgrade the Java runtime environment of the EC2 instances in the Elastic Beanstalk environment.

Correct

Elastic Beanstalk regularly releases new platform versions to update all Linux-based and Windows Server-based platforms. New platform versions provide updates to existing software components and support for new features and configuration options.

You can use the Elastic Beanstalk console or the EB CLI to update your environment’s platform version. Depending on the platform version you’d like to update to, Elastic Beanstalk recommends one of two methods for performing platform updates.

Method 1 – Update your Environment’s Platform Version – This is the recommended method when you’re updating to the latest platform version, without a change in runtime, web server, or application server versions, and without a change in the major platform version. This is the most common and routine platform update.

Method 2 – Perform a Blue/Green Deployment – This is the recommended method when you’re updating to a different runtime, web server, or application server version or to a different major platform version. This is a good approach when you want to take advantage of new runtime capabilities or the latest Elastic Beanstalk functionality.

Because AWS Elastic Beanstalk performs an in-place update when you update your application versions, your application can become unavailable to users for a short period of time. You can avoid this downtime by performing a blue/green deployment, where you deploy the new version to a separate environment and then swap CNAMEs of the two environments to redirect traffic to the new version instantly.

Blue/green deployments require that your environment runs independently of your production database if your application uses one. If your environment has an Amazon RDS DB instance attached to it, the data will not transfer over to your second environment and will be lost if you terminate the original environment.

Hence, the correct answer is to **perform a Blue/Green deployment** to safely upgrade the application's runtime environment from Java 7 to Java 8.

The option that says: **Update the environment's platform version to Java 8** is incorrect because using this method is only recommended when you're updating to the latest platform version without a change in the runtime environment.

The option that says: **Manually upgrade the Java runtime environment of the EC2 instances in the Elastic Beanstalk environment** is incorrect. Although this method will work, this entails a lot of configuration to implement compared with just performing a blue/green deployment. In addition, this method may introduce operational risk because the environment may go down while the developer is doing the updates manually.

The option that says: **Perform a Traffic splitting deployment** is incorrect because the scenario requires all user traffic to be immediately directed towards the new version. Take note that in Traffic splitting, updates are released incrementally to a subset of users.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.platform.upgrade.html#using-features.platform.upgrade.bluegreen>
<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.CNAMEswap.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdojo.com/aws-elastic-beanstalk/>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-developer-associate/>

20. Question

Category: CDA – Deployment

You developed a shell script which uses AWS CLI to create a new Lambda function. However, you received an `InvalidParameterValueException` after running the script.

What is the MOST likely cause of this issue?

- The resource already exists.
- The AWS Lambda service encountered an internal error.
- You provided an IAM role in the `CreateFunction` API which AWS Lambda is unable to assume.
- You have exceeded your maximum total code size per account.

Incorrect

To create a Lambda function, you need a deployment package and an execution role. The deployment package contains your function code. The execution role grants the function permission to use AWS services, such as Amazon CloudWatch Logs for log streaming and AWS X-Ray for request tracing. You can use the `CreateFunction` API via the AWS CLI or the AWS SDK of your choice.

A function has an unpublished version, and can have published versions and aliases. The unpublished version changes when you update your function's code and configuration. A published version is a snapshot of your function code and configuration that can't be changed. An alias is a named resource that maps to a version, and can be changed to map to a different version.

The `InvalidParameterValueException` will be returned if one of the parameters in the request is invalid. For example, if **you provided an IAM role in the `CreateFunction` API which AWS Lambda is unable to assume**. Hence, this option is the most likely cause of the issue in this scenario.

If **you have exceeded your maximum total code size per account**, the `CodeStorageExceededException` will be returned, which is why this option is incorrect.

If **the resource already exists**, the `ResourceConflictException` will be returned and not `InvalidParameterValueException`. Therefore, this option is also incorrect.

If **the AWS Lambda service encountered an internal error**, the `ServiceException` will be returned hence, this option is incorrect.

References:

https://docs.aws.amazon.com/lambda/latest/dg/API_CreateFunction.html
<https://docs.aws.amazon.com/cli/latest/reference/lambda/create-function.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

21. Question

Category: CDA – Deployment

A developer is writing a CloudFormation template which will be used to deploy a simple Lambda function to AWS. The function to be deployed is made in Python with just 3 lines of codes which can be written inline in the template.

Which parameter of the `AWS::Lambda::Function` resource should the developer use to place the Python code in the template?

- `CodeUri`
- `Handler`
- `ZipFile`
- `Code`

Correct

To create a Lambda function you first create a Lambda function deployment package, a .zip or .jar file consisting of your code and any dependencies. When creating the zip, include only the code and its dependencies, not the containing folder. You will then need to set the appropriate security permissions for the zip package.

If you are using a CloudFormation template, you can configure the `AWS::Lambda::Function` resource which creates a Lambda function. To create a function, you need a deployment package and an execution role.

The deployment package contains your function code. The execution role grants the function permission to use AWS services, such as Amazon CloudWatch Logs for log streaming and AWS X-Ray for request tracing.

ManagedPolicyArns:

- `arn:aws:iam::aws:policy/AWSLambdaExecute`
- `arn:aws:iam::aws:policy/AmazonS3FullAccess`
- `arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess`
- `arn:aws:iam::aws:policy/AmazonKinesisFullAccess`

Path: /

LambdaFunction:

Type: `AWS::Lambda::Function`

Properties:

FunctionName:

Fn::Sub: `lambda-function-${EnvName}`

Description: LambdaFunction1 of node 8

Runtime: `nodejs8.10`

Code:

ZipFile:

```
"exports.handler = function(event, context){\n
  console.log('Hello World By Tutorials Dojo');\n
}"
```

Handler: `${LambdaHandlerPath}`

MemorySize: 128

Timeout: 10

Role:

- Fn::GetAtt:**
- `LambdaRole`
- `Arn`

Environment:

Variables:

ENV:

Fn::Sub: `${EnvName}`

TZ: UTC

Under the `AWS::Lambda::Function` resource, you can use the `Code` property which contains the deployment package for a Lambda function. For all runtimes, you can specify the location of an object in Amazon S3. For Node.js and Python functions, you can specify the function code inline in the template. Changes to a deployment package in Amazon S3 are not detected automatically during stack updates. To update the function code, change the object key or version in the template.

Hence, the `ZipFile` parameter to is the correct one to be used in this scenario, which will allow the developer to place the python code inline in the template. If you include your function source inline with this parameter, AWS CloudFormation places it in a file named `index` and `zips` it to create a deployment package. This is the reason why it is called the "ZipFile" parameter, and not because it accepts zip files.

The `Handler` parameter is incorrect because this is not a valid property of `AWS::Lambda::Function` resource but of the `AWS::Serverless::Function` resource in AWS SAM. In addition, this parameter is

primarily used to specify the name of the handler, which is just a function in your code that AWS Lambda can invoke when the service executes your code. The **Code** parameter is incorrect because you should use the **ZipFile** parameter instead. Take note that the **Code** property is the parent property of the **ZipFile** parameter. The **CodeUri** parameter is incorrect because this is not a valid property of **AWS::Lambda::Function** resource but of the **AWS::Serverless::Function** resource in AWS SAM. This parameter accepts the S3 URL of your code and not the actual code itself.

References:
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-properties-lambda-function-code.html>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-lambda-function.html>

Check out this AWS CloudFormation Cheat Sheet:
<https://tutorialsdojo.com/aws-cloudformation/>

22. Question

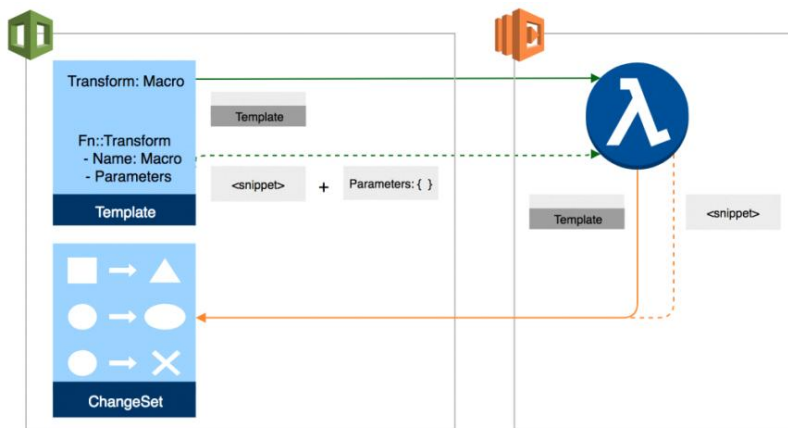
Category: CDA – Deployment

You are using AWS Serverless Application Model (AWS SAM) to build and deploy applications in your serverless infrastructure. Your manager instructed you to create a CloudFormation template that includes your SAM script and other service configurations. This template will be used to launch a similar infrastructure in another region.

What should you do in order to accomplish this task?

- Add a **Resources** section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use.
- Add a **Mappings** section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use.
- Add a **Parameters** section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use.
- Add a **Transform** section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use.

Correct
 AWS CloudFormation is a service that helps you model and set up your Amazon Web Services resources so that you can spend less time managing those resources and more time focusing on your applications that run in AWS. You create a template that describes all the AWS resources that you want, and AWS CloudFormation takes care of provisioning and configuring those resources for you. A CloudFormation template is a JSON- or YAML-formatted text file that describes your AWS infrastructure. The template includes several sections for you to define your infrastructure code. For serverless applications (also referred to as Lambda-based applications), the **Transform** section specifies the version of the AWS Serverless Application Model (AWS SAM) to use. When you specify a transform, you can use AWS SAM syntax to declare resources in your template. The model defines the syntax that you can use and how it is processed. More specifically, the **AWS::Serverless** transform, which is a macro hosted by AWS CloudFormation, takes an entire template written in the AWS Serverless Application Model (AWS SAM) syntax and transforms and expands it into a compliant AWS CloudFormation template.



In the following example, the template uses AWS SAM syntax to simplify the declaration of a Lambda function and its execution role.

```
Transform: AWS::Serverless-2016-10-31
Resources:
  MyServerlessFunctionLogicalID:
    Type: AWS::Serverless::Function
    Properties:
      Handler: index.handler
      Runtime: nodejs8.10
      CodeUri: 's3://testBucket/mySourceCode.zip'
```

Hence, the correct answer is: **Add a Transform section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use.**

The option that says: **Add a Parameters section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use** is incorrect because this is just the part of the template that contains values to pass to your template at runtime when you create or update a stack.

The option that says: **Add a Resources section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use** is incorrect because this is primarily used to specify the stack resources and their properties.

The option that says: **Add a Mappings section in the template to specify the version of the AWS Serverless Application Model (AWS SAM) to use** is incorrect because this just lists a mapping of keys and associated values that you can use to specify conditional parameter values, similar to a lookup table. You can match a key to a corresponding value by using the **Fn::FindInMap** intrinsic function in the **Resources** and **Outputs** sections.

References:
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-anatomy.html>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/transform-aws-serverless.html>

Check out this AWS CloudFormation Cheat Sheet:
<https://tutorialsdojo.com/aws-cloudformation/>

Check out this AWS Serverless Application Model Cheat Sheet:
<https://tutorialsdojo.com/aws-serverless-application-model-sam/>

23. Question

Category: CDA – Deployment

A developer is building an e-commerce application which will be hosted in an ECS Cluster. To minimize the number of instances in use, she must select a strategy which will place tasks based on the least available amount of CPU or memory.

Which of the following task placement strategy should the developer implement?

- random
- binpack
- distinctInstance
- spread

Correct
 A **task placement strategy** is an algorithm for selecting instances for task placement or tasks for termination. Task placement strategies can be specified when either running a task or creating a new service.

Amazon ECS supports the following task placement strategies:

binpack – Place tasks based on the least available amount of CPU or memory. This minimizes the number of instances in use.

random – Place tasks randomly.

spread – Place tasks evenly based on the specified value. Accepted values are attribute key-value pairs, *instanceId*, or *host*.

The scenario states that the developer must select a task placement strategy which will place tasks based on the least available amount of CPU or memory. By using bin pack strategy with CPU as the field parameter, ECS is able to place tasks onto an instance with the least available amount of CPU first, before moving on to the other instances. Hence, the correct answer is to use the **binpack** task placement strategy.

random is incorrect because this will place the tasks randomly, rather than placing the tasks to the instances based on the least available amount of CPU or memory.

spread is incorrect because this will place tasks evenly to the instances based on a specified value.

distinctInstance is incorrect because this is not a valid task placement strategy, but a task placement constraint. This is primarily used as a constraint to place each task on a different container instance. It can be specified when either running a task or creating a new service.

References:

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement.html>

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement-strategies.html>

<https://aws.amazon.com/blogs/compute/amazon-ecs-task-placement/>

Check out this Amazon ECS Cheat Sheet:

<https://tutorialsdoido.com/amazon-elastic-container-service-amazon-ecs/>

24. 24. Question

Category: CDA – Deployment

A single docker container environment is hosted in Elastic Beanstalk. Your manager instructed you to ensure that the compute resources maintain full capacity during deployments to avoid any degradation of the service or possible down time.

Which of the following deployment methods should you use to satisfy the given requirement? (Select TWO.)

- Rolling
- Canary
- Rolling with additional batch
- Immutable
- All at once

Correct

In ElasticBeanstalk, you can choose from a variety of deployment methods:

All at once – Deploy the new version to all instances simultaneously. All instances in your environment are out of service for a short time while the deployment occurs.

Rolling – Deploy the new version in batches. Each batch is taken out of service during the deployment phase, reducing your environment's capacity by the number of instances in a batch.

Rolling with additional batch – Deploy the new version in batches, but first launch a new batch of instances to ensure full capacity during the deployment process.

Immutable – Deploy the new version to a fresh group of instances by performing an immutable update.

Blue/Green – Deploy the new version to a separate environment, and then swap CNAMEs of the two environments to redirect traffic to the new version instantly.

Method	Impact of Failed Deployment	Deploy Time	Zero Downtime	No DNS Change	Rollback Process	Code Deployed To
All at once	Downtime	☹	✗	✓	Manual Redeploy	Existing instances
Rolling	Single batch out of service; any successful batches before failure running new application version	☹ ☹ ☹ ↑	✓	✓	Manual Redeploy	Existing instances
Rolling with additional batch	Minimal if first batch fails; otherwise, similar to Rolling	☹ ☹ ☹ ☹ ↑	✓	✓	Manual Redeploy	New and existing instances
Immutable	Minimal	☹ ☹ ☹ ☹ ☹	✓	✓	Terminate New Instances	New instances
Blue/green	Minimal	☹ ☹ ☹ ☹ ☹	✓	✗	Swap URL	New instances

To maintain full capacity during deployments, you can configure your environment to launch a new batch of instances before taking any instances out of service. This option is known as a **rolling deployment with an additional batch**. When the deployment completes, Elastic Beanstalk terminates the additional batch of instances.

Immutable environment updates are an alternative to rolling updates. Immutable environment updates ensure that configuration changes that require replacing instances are applied efficiently and safely. If an immutable environment update fails, the rollback process requires only terminating an Auto Scaling group. A failed rolling update, on the other hand, requires performing an additional rolling update to roll back the changes.

To perform an immutable environment update, Elastic Beanstalk creates a second, temporary Auto Scaling group behind your environment's load balancer to contain the new instances. First, Elastic Beanstalk launches a single instance with the new configuration in the new group. This instance serves traffic alongside all of the instances in the original Auto Scaling group that are running the previous configuration.

Hence, **Immutable** and **Rolling with additional batch** are the correct deployment methods to be used in this scenario.

All at once is incorrect because this will deploy the new version to all existing instances immediately and will not create new EC2 instances. Hence, it is possible that there would be a degradation of the service since some instances would be unavailable during the deployment process.

Rolling is incorrect because this will deploy the new version in batches only to existing instances, without provisioning new resources. The compute capacity of the environment would still be compromised in this method. **Canary** is incorrect because this type of deployment method is not readily available in Elastic Beanstalk, but primarily to Lambda.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environmentmgmt-updates-immutable.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdoido.com/aws-elastic-beanstalk/>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdoido.com/aws-certified-developer-associate/>

25. 25. Question

Category: CDA – Deployment

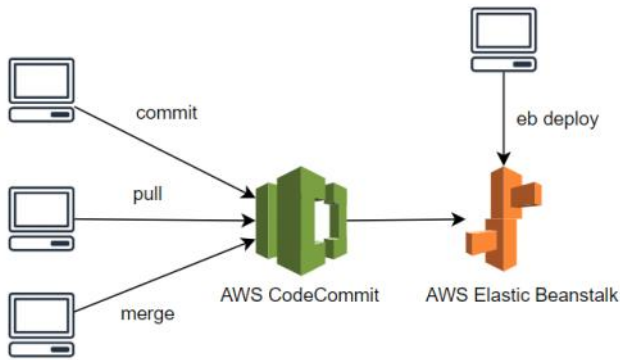
Several development teams worldwide will be collaboratively working on a project hosted on an AWS Elastic Beanstalk environment. The developers need to be able to deploy incremental code updates without re-uploading the entire project.

Which of the following actions will reduce the upload and deployment time with the LEAST amount of effort?

- Host the code repository on an EC2 instance and allow access to all the developers. Write a script that will automate the deployment process to Elastic Beanstalk.
- Configure event notifications on a central S3 bucket and allow access to all developers. Invoke a Lambda Function that will deploy the code to Elastic Beanstalk when a PUT event occurs.
- Create an AWS CodeCommit repository and allow access to all developers. Deploy the code to Elastic Beanstalk.
- Upload the code to an Amazon EFS mounted on an EC2 instance. Write a script that will automate the deployment process to Elastic Beanstalk.

Correct

CodeCommit is a secure, highly scalable, managed source control service that hosts private Git repositories. CodeCommit eliminates the need for you to manage your own source control system or worry about scaling its infrastructure.



AWS CodeCommit is designed for collaborative software development. You can easily commit, branch, and merge your code allowing you to easily maintain control of your team's projects. CodeCommit also supports pull requests, which provide a mechanism to request code reviews and discuss code with collaborators. You can create a repository from the AWS Management Console, AWS CLI, or AWS SDKs and start working with the repository using Git.

You can use the EB CLI to deploy your application directly from your AWS CodeCommit repository. With CodeCommit, you can upload only your changes to the repository when you deploy, instead of uploading your entire project.

Hence, the correct answer is: **Create an AWS CodeCommit repository and allow access to all developers. Deploy the code to Elastic Beanstalk.**

The option that says: **Host the code repository on an EC2 instance and allow access to all the developers. Write a script that will automate the deployment process to Elastic Beanstalk** is incorrect because this solution entails a lot of effort — from selecting the AMI to managing the instance, setting up a code repository, and configuring scalability. AWS CodeCommit is already a managed and highly scalable source control service that has native integration with Elastic Beanstalk.

The option that says: **Configure event notifications on a central S3 bucket and allow access to all developers. Invoke a Lambda Function that will deploy the code to Elastic Beanstalk when a PUT event occurs** is incorrect. Amazon S3 does not have the functionality to modify stored objects. Incremental code updates are not possible with S3, which defies the requirement for the scenario.

The option that says: **Upload the code to an Amazon EFS mounted on an EC2 instance. Write a script that will automate the deployment process to Elastic Beanstalk** is incorrect. Although EFS is a good use case for concurrent access, it is not a suitable solution for a source control service. EFS is less performant to workloads that require random access over large files. Use EFS if you want to distribute highly parallelized workloads like analytical workloads and media processing across several machines.

References:

<https://aws.amazon.com/premiumsupport/knowledge-center/deploy-codecommit-elastic-beanstalk/>
<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/eb-cli-codecommit.html>

Check out these AWS CodeCommit and AWS Elastic Beanstalk Cheat Sheets:

<https://tutorialsdojo.com/aws-elastic-beanstalk/>
<https://tutorialsdojo.com/aws-codecommit/>

26. Question

Category: CDA – Deployment

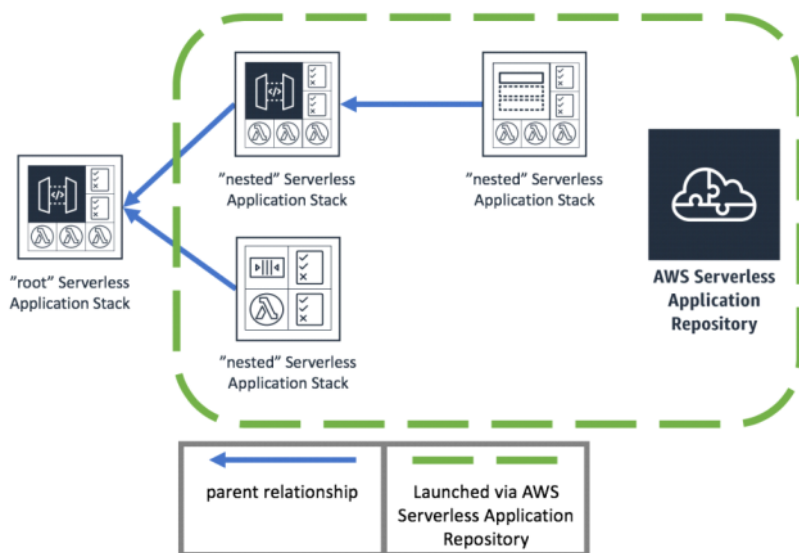
A development team has recently completed building their serverless application. They must zip their code artifacts, upload them to Amazon S3, produce the package template file for deployment, and deploy it to AWS. Which command is the MOST suitable to use to automate the deployment steps?

- `sam package`
- `aws cloudformation deploy`
- `sam deploy`
- `sam publish`

Incorrect

AWS SAM uses AWS CloudFormation as the underlying deployment mechanism. You can deploy your application by using AWS SAM command line interface (CLI) commands. You can also use other AWS services that integrate with AWS SAM to automate your deployments.

After you develop and test your serverless application locally, you can package and deploy your application by using the `sam deploy` command.



The `sam deploy` command zips your code artifacts, uploads them to Amazon S3, and produces a packaged AWS SAM template file that it uses to deploy your application.

To deploy an application that contains one or more nested applications, you must include the `CAPABILITY_AUTO_EXPAND` capability in the `sam deploy` command.

Hence, the correct answer is: **`sam deploy`**

`aws cloudformation deploy` is incorrect. While this command can be used to deploy a CloudFormation stack, it expects that your artifacts are already packaged and uploaded to S3. It doesn't handle the packaging process implicitly.

`sam package` is incorrect. This command simply prepares the serverless application for deployment by zipping artifacts, uploading them to S3, and generating a CloudFormation template with references to the uploaded artifacts in S3. It doesn't deploy the application.

`sam publish` is incorrect because this command publishes an AWS SAM application to the AWS Serverless Application Repository and does not generate the template file. It takes a packaged AWS SAM template and publishes the application to the specified region.

References:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-deploying.html>
<https://docs.aws.amazon.com/serverlessrepo/latest/devguide/what-is-serverlessrepo.html>

Check out this AWS SAM Cheat Sheet:

<https://tutorialsdodo.com/aws-serverless-application-model-sam/>

From <<https://portal.tutorialsdodo.com/courses/aws-certified-developer-associate-practice-exams/lessons/practice-exams-section-based-3/quizzes/aws-certified-developer-associate-practice-exam-section-based-deployment/>>